

All About the Challenges

AMD AI Challenges

Who this is for

You've joined the AMD AI Challenge. This document tells you everything you need to build, run, and submit a solution: where to get a GPU, what your submission must look like, and exactly how it will be scored.

Read Section 2 before you start building. Most lost points are not lost on model quality — they're lost on packaging.

The challenges run every two weeks from 1 September to 1 December. The infrastructure and submission rules here are the same for every challenge; only the task changes. Challenge-specific details are at the end.

Section 1 — Infrastructure

Before you start

Four steps, in this order. All four are needed before you can run anything.

1. Join the AMD AI Developer Program — <https://www.amd.com/en/developer/ai-dev-program.html>
2. Create an AMD Developer Cloud account
3. Complete the Build AI learning pathway on AMD AI Academy — <https://developer.amd.com/amd-ai-academy/>
4. Claim your cloud credits

The AI Academy link above is the one to use — completing a learning pathway there is Mini-Challenge 1, and course completions are what earn your Learning points.

Getting access to a GPU

Important: You do not need your own AMD hardware, AMD Developer Cloud credits, or Fireworks API credits to take part in a challenge. Please read the following paragraph on how you can access GPUs during mini challenges.

GPU Infrastructure can be accessed at <https://notebooks.amd.com/hackathon>. You will need to sign-in via AMD Dev Program SSO credentials and the first launch can be done pressing "Launch Notebook" button on the page. Launching one gives you a JupyterLab session on a GPU node: you start the notebook, wait for it to become ready, and open it. The first redirected URL when you open a session is single-use and short-lived, so open your session from a fresh tab and visiting <https://notebooks.amd.com/hackathon> each time rather than bookmarking the URL.

Your challenge session runs on the dedicated hackathon pool with these resources.

Resource	What you get
GPU	1 GPU per session – Can be CDNA or RDNA
Memory	48 GiB system VRAM
Notebook	Pre-built into the session image — nothing to clone
Persistent storage	Maximum 25 GB per user

Time quota

Sessions are capped at 3 hours of running time each day. Every 24 hours the quota refreshes. A session left idle still consumes that quota, so press the "Turn-off Session" button — by opening another <https://notebooks.amd.com/hackathon> in a new tab of your browser — when you stop working. The quota is time the pod exists, not time you spend typing.

You get a fresh 3 hours of quota each day. Unused hours do NOT roll over to the next day.

GPUs are limited and are allocated on a first-come, first-served basis. If you see a “No GPUs Available” message, please try again in a few hours.

Storage

Each user gets around 25gb of persistent storage. **The location of persistent storage depends on the pod url (very important):**

- If your redirected URL name has “jupyter-hack-***” in it then your persistent storage will be at /persistent and only files/folders up to about 25gb stored in /persistent location of your jupyter-lab environment survives quota exhaustion and turn-off session. Files stored under /workspace will not be persisted for those users/teams
- If your redirected URL has “rgapi-hackathon-***” in it then our persistent storage will be at /workspace and only files/folders up to about 25gb stored in /workspace location of your jupyter-lab environment survives quota exhaustion and turn-off session. Files stored under any other location will not be persisted for those users/teams

Base images and dependencies

Your challenge session comes with its environment already built in — the notebook and its dependencies are part of the session image, so there is nothing to clone or install to get started.

The session image is built from the same ROCm base image your submission must be built on: rocm/pytorch:rocm10.0_ubuntu26.04_py3.14_pytorch_release_2.13.0. What you develop in and what you are graded on start from the same place.

That base image ships torch and PIL only — no transformers, no flash-attn, no vLLM. Install what you need with a requirements.txt. Note that a pip install done inside a session lasts only for that session: when your daily quota resets and you open a new one, you will need to run it again.

Office hours and getting help

The challenge Discord is the fastest route to an answer, and constructive participation there earns points in its own right — answering other participants' questions counts toward your score.

The hackathon page — <https://lablab.ai/ai-hackathons/amd-lablab-ai-academy-challenge> — carries the current schedule and announcements.

Office hours run 11:00 AM – 1:00 PM Eastern Time.

Section 2 — Evaluation and Submission

How evaluation works

Your solution is packaged as a Docker container and run on an AMD GPU. For each test image, our harness runs your script inside your container, passing the image path as an argument, and reads the JSON file your script writes. Your code owns the whole pipeline — loading the model, any pre-processing, detection, and any post-processing.

Three things are checked before your solution is scored at all:

- Your image is built on the mandated ROCm base image.
- Your image is within the size limit.
- Your container starts, and every image is processed, within the time limits.

If any of these fails the submission scores zero. These are hard gates, not deductions, and all three are things you can verify yourself in advance — see “Check your submission before you send it”.

The test images are hidden. The sample images at the end of this document show you the formats you must accept and the transcription convention we expect — they are not the images we grade with, and the graded set is deliberately harder. A solution that hardcodes answers to the samples scores zero.

Container requirements

Base layer must be ROCm. Your image must be built on the ROCm base image we mandate — the same image your development session runs. This is checked automatically before your container is run, and it is checked by layer identity: we compare the lower layers of your image against the base itself, not the name you gave it. A submission that is not built on it is **disqualified**. Two consequences worth knowing. A multi-stage build is fine, as long as the FINAL stage starts from the mandated base. But squashing or flattening your image — `docker build --squash`, `buildah`, or copying everything into a FROM scratch layer — destroys the layer identity we verify against, and the submission is rejected even though you did build on the right base. If you are squashing to get under the size limit, do not: put your dependencies in a `requirements.txt` instead of baking them into extra layers.

```
FROM rocm/pytorch:rocm10.0_ubuntu26.04_py3.14_pytorch_release_2.13.0
# the base we mandate: it supports both MI300X and Radeon
```

Maximum image size: 60 GiB, measured uncompressed.

Model weights are your choice. You can ship them inside the image, which counts against the size limit, or have your application download them at run time — your container has outbound network access during evaluation. We do not download anything for you: if your solution fetches weights, your own code must do it and place them where your pipeline expects to read them.

Measure the size of your Docker image uncompressed. Going over the limit is an automatic zero, so do check before you submit.

In your Docker container, your code must sit at these exact paths — the harness reads them, and this is what it enforces:

Path	What goes there
/app	Your application code, including app.py (see “How your code is called”)
/app/requirements.txt	Python dependencies, if you use them
/models	Model weights you ship with the image (optional)

The paths matter. If /app/app.py is not there, or it does not accept the arguments below, we cannot run your solution and it scores zero.

Hardware and model constraints

Your submission must use between 1 GiB and 48 GiB of VRAM, with a margin of 1% allowed on the upper bound; outside that range the submission scores zero. We sample VRAM every 3 seconds while your solution runs, and the peak is what counts. A submission that never uses the GPU has not solved the problem we set.

You may use any open-source model that runs on AMD GPU hardware. We deliberately do not prescribe a model, a serving framework, or an inference library. Choosing well is part of the challenge.

Never put secrets in your image

Your image is public, and we pull it. So can everybody else.

No .env file, no API keys, no tokens, no cloud credentials, in the image or in the repository

How is your code called?

There is no service to run and no endpoint to expose. For each test image, we execute your script inside your already-running container:

```
python3 /app/app.py --input-image /app/input/image_01.png
```

Your script reads the image at the path given, does whatever it needs to do, and writes its answer as a JSON file into /app/output/. The output filename is the input filename with _output.json in place of its extension — so image_01.png produces /app/output/image_01_output.json. Each image gets its own output file, so there is no ambiguity about which answer belongs to which image.

Expected output — /app/output/image_01_output.json:

```
{  
  "text": "7ABC123",  
  "confidence": 0.94  
}
```

Field	Type	Required	Notes
text	string	yes	The characters read from the image, exactly as they appear — including Chinese characters where the image contains them. This is what we grade.
confidence	float 0.0–1.0	no	Your own confidence. Recorded, not scored.

Time limits

Limit	Value	What it covers
Startup	10 minutes	Container start
Per image	30 seconds	Each inference call
Total run	10 minutes	All 10 test images, after startup has completed

Exceeding the startup budget or any single image budget ends the run and scores zero. Load your model once, not once per image.

Accepted image formats

Test images will be PNG, JPEG or TIFF. Your solution must accept all three.

Image dimensions are not constrained. Expect a range of sizes and aspect ratios, including images larger than your model's native input resolution. Resizing, preprocessing and postprocessing are your responsibility.

How you are scored?

There are 10 test images, each worth 20 points. Maximum 200 points.

Each image is graded pass/fail: your text either matches the expected answer after normalization, or it does not.

The three hard gates above still apply: a wrong base image, exceeding the size limit, or exceeding a time limit scores zero overall.

Normalization — read this carefully

Before comparison, both your answer and the expected answer are normalized:

- Converted to uppercase
- All whitespace removed
- The characters - . · _ removed

So for a plate reading 7ABC123, all of these are correct:

7ABC123 7abc123 7-ABC-123 7 ABC 123

And these are wrong:

7ABC128 (misread character)
 CALIFORNIA 7ABC123 (state name is not part of the plate number)
 PLATE: 7ABC123 (extra words)

Return only the characters present in the image, in reading order. Do not add labels, prefixes, units or explanation. Where text runs across multiple lines, read top to bottom and join the lines with a single space.

One important qualification on "not the state name". What you drop is text printed AROUND the number — a jurisdiction banner like CALIFORNIA or TEXAS, or a slogan like THE LONE STAR STATE. On a Chinese plate the leading province character and letter are not a banner; they are the first two characters OF the registration. 京 A·12345 is transcribed 京 A·12345, not 12345. The rule is "drop what is printed around the number", not "drop the province".

CALIFORNIA 7ABC123 -> 7ABC123 banner, dropped
 京 A·12345 -> 京 A·12345 part of the number, kept

The sample images below are the reference for how any given image should be transcribed.

Check your submission before you send it

Check these locally before you submit. They are the same checks our pipeline runs, and if any of them fails on your machine it will fail on ours.

1. Your image is built on the mandated ROCm base, and is under the size limit uncompressed:

```
docker history --no-trunc --format '{{.Size}}' your-image:tag
```

2. Your container starts inside the startup budget.

3. `python3 /app/app.py --input-image <path>` runs to completion inside the per-image budget, for a PNG, a JPEG and a TIFF.

4. It writes `/app/output/<name>_output.json` in exactly the format above, for every image.

5. VRAM stays between 1 GiB and 48 GiB while it runs, not just at rest:

```
watch -n1 'amd-smi metric --mem | grep USED_VRAM' # or: rocm-smi --showmeminfo vram
```

How to submit

Push your built image to a public registry and submit the image reference. It must be pullable without credentials.

We pull your image and run it as-is — nothing is built at evaluation time, so what you test locally is exactly what we score. If your image is private, or the reference does not resolve, the submission cannot be scored. One thing to be careful about: your image has to be publicly pullable for us to fetch it, but do not advertise where it lives. Do not commit the image reference to a public repository. There is no naming convention you must follow.

Mini-Challenge 2 — Optical Character Recognition

Opens 14 September 2026

Why this matters

Surveillance systems use AI to read license plates. Autonomous driving systems read speed limit postings and advisories — road work ahead, icy bridge, detour. Both are the same underlying problem: pulling reliable text out of an uncooperative image.

What you are building

An application that accepts an image and uses a vision model or vision-language model to recognize the characters in it and report what it found. How you build it is open, as long as it meets the invocation contract in Section 2. Your end-to-end solution is packaged as a Docker container per Section 2.

You will be tested on:

- US licence plates — several state designs, with state names and slogans printed alongside the plate number
- Stop signs
- Speed limit signs, and advisory speed plaques that carry a number and no words
- Work zone and warning signs — multi-line text
- All of the above under adverse conditions — noise, blur, glare, low light, off-axis angles

Build for generality. The graded set spans all of these categories and is harder than the samples below.

Sample images and expected responses

These ten images show the formats you must accept and the transcription convention we expect. They are illustrative — the graded images are different.

1 — US plate (California), clean · PNG



Expected response: 7ABC123

Baseline. Return the plate number only — not the state name.

2 — Chinese plate (Beijing), clean · PNG



Expected response: 京 A·12345

The leading province character and letter are part of the registration, not a banner printed around it — so 京 A is kept, and the answer is 京 A·12345 rather than 12345. Contrast sample 1, where CALIFORNIA is dropped. The interpunct · is removed by normalization, so 京 A12345 is equally correct.

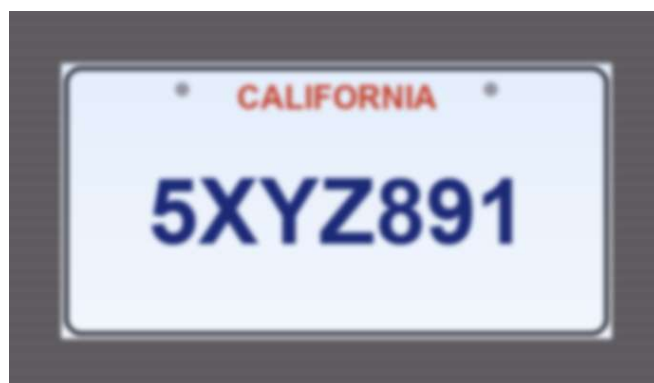
3 — US plate (New York), angled · JPEG



Expected response: JHT 2951

Perspective skew, as from an off-axis camera.

4 — US plate, motion blur · PNG



Expected response: 5XYZ891

Defocus and motion blur, as from a moving vehicle.

5 — Chinese plate (Shanghai), low light with glare · JPEG



Expected response: 沪 B·88888

Under-exposed with glare across the plate face. Same province rule as sample 2: 沪 B is part of the number. Low contrast plus a repeated digit makes this the hardest plate in the set.

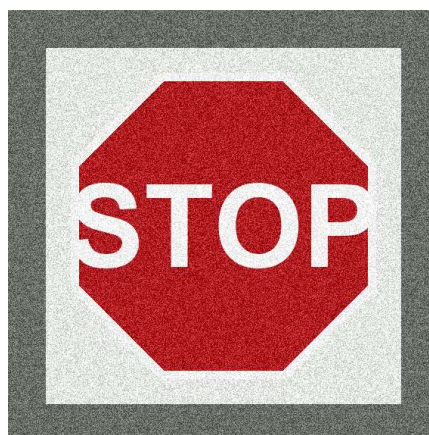
6 — Stop sign, clean · PNG



Expected response: STOP

Single word.

7 — Stop sign, sensor noise · TIFF



Expected response: STOP

Heavy additive noise. The answer is unchanged by it.

8 — Speed limit sign · JPEG



Expected response: SPEED LIMIT 65

The words are printed on the sign, so they are part of the answer.

9 — Work zone sign, multi-line · PNG



Expected response: ROAD WORK AHEAD

Three lines. Read top to bottom and join with single spaces.

10 — Advisory speed plaque • TIFF



Expected response: 35

Digits only. The sign carries no unit, so do not add one.

Useful Links

- OCR with vLLM on ROCm — https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/inference/ocr_vllm.html
- Host Your First LLM on AMD GPU — <https://lablab.ai/ai-tutorials/amd-developer-cloud-host-llm-vllm>

We are not recommending a specific OCR or vision framework. Several open-source options run well on AMD hardware and evaluating them is part of the work.